MP3: Page Manager I

Phani Jyothi Kurada
UIN: 335006950
CSCE611: Operating System

Assigned Tasks

Main: Completed.

System Design

The objective of this project is to implement a **demand-paging virtual memory system** within the kernel. The primary goal is to set up the paging mechanism and configure the paging subsystem for a single address space environment.

Memory Management Structure

The memory management architecture is organized into two hierarchical levels:

- **Level 1: Page Directory** – Contains pointers to individual page tables.
- **Level 2: Page Tables** – Each page table contains entries mapping virtual pages to physical frames.

The system uses a 32-bit virtual address, which is divided into three parts as follows:

Bits [31:22]	**Bits [21:12]**	**Bits [11:0]**
Page Directory Index (10 bits)	Page Table Index (10 bits)	Offset (12 bits)

Address Translation Process

The virtual-to-physical address translation is performed in three stages:

1. The *Page Directory Index* selects the corresponding Page Directory Entry (PDE), which points to the base address of the required page table.
2. The *Page Table Index* selects the Page Table Entry (PTE) within that page table, which holds the frame number of the target physical memory page.
3. The *Offset* (lower 12 bits) is added to the frame base address to obtain the final physical address.

Design Rationale

This two-level paging mechanism provides a balance between efficient memory utilization and ease of management. By dividing the virtual address space hierarchically, the system supports large address spaces while minimizing the amount of memory reserved for page tables. It also facilitates the implementation of demand paging, where pages are allocated dynamically as needed, reducing unnecessary memory usage and improving overall system performance.

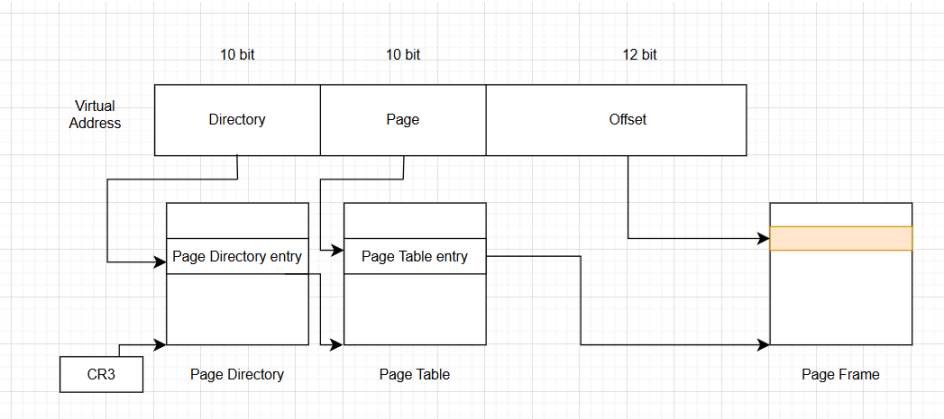


Figure 1: 2 - Level Paging Scheme

Code Description

This section describes the key classes and functions implemented to support the demand-paging virtual memory system. The main components involved in this module include the `ContFramePool` class and the `PageTable` class. The system integrates the professor-provided `cont_frame_pool.H` and `cont_frame_pool.o` files for frame management.

ContFramePool Class

The `ContFramePool` class, provided via `cont_frame_pool.H`, is responsible for managing physical memory frames within a predefined pool. It maintains an internal bitmap to track the status of each frame. Each frame can be in one of three states:

- **Free:** The frame is available for allocation.
- **Used:** The frame is currently allocated to a process or kernel structure.
- **Head of Sequence (HOS):** The frame represents the starting frame of a contiguous sequence of allocated frames.

This class supports allocation of contiguous sequences of frames, which are necessary for certain kernel structures and page tables.

PageTable Class : `page_table.C`

The `PageTable` class manages the virtual-to-physical address mapping in the system. It provides methods for initializing, loading, and enabling paging within the kernel also for handling page faults when occured.

- **init_paging():** Sets up the paging subsystem by initializing the kernel and process memory pools and defining the shared memory size. This function establishes the global structures required for paging before any virtual-to-physical address translation occurs. It also ensures that subsequent allocations for page tables and frames use the correct memory pools.

```
/*--------------------------------------------------------------*/
/* Initialize the global paging system parameters */
/*--------------------------------------------------------------*/
void PageTable::init_paging(ContFramePool * _kernel_mem_pool,
                            ContFramePool * _process_mem_pool,
                            const unsigned long _shared_size)
{
    PageTable::kernel_mem_pool = _kernel_mem_pool;
    PageTable::process_mem_pool = _process_mem_pool;
    PageTable::shared_size = _shared_size;
    Console::puts("Initialized Paging System\n");
}
```

Figure 2: `init_paging()` Implementation

- **Constructor:** `PageTable()` Allocates memory for the Page Directory and the first Page Table. It identity-maps the initial 4 MB region (or up to the defined shared memory size) so that virtual addresses in this region directly correspond to physical addresses. The constructor calculates the number of frames to map, allocates frames from the kernel memory pool, and initializes all entries in the page table with present and writable flags. All other Page Directory entries are marked as not present initially. This prepares the structures required for address translation and ensures the first memory region is accessible immediately.

```
/*--------------------------------------------------------------*/
/* Construct a page table */
/*--------------------------------------------------------------*/
PageTable::PageTable()
{
    // Calculate number of frames to identity-map (up to 4 MB)
    unsigned long total_frames = shared_size / PAGE_SIZE;
    if (total_frames > 1024) total_frames = 1024;

    // Allocate one frame for page directory
    page_directory = (unsigned long *)(kernel_mem_pool->get_frames(1) * PAGE_SIZE);

    // Allocate one frame for the first page table (4 MB region)
    unsigned long * page_table = (unsigned long *)(kernel_mem_pool->get_frames(1) * PAGE_SIZE);

    // Identity map the first 4MB (virtual = physical)
    unsigned long phys_addr = 0;
    for (unsigned int i = 0; i < total_frames; i++) {
        page_table[i] = (phys_addr) | 3; // Present + Writable
        phys_addr += PAGE_SIZE;
    }

    // Initialize all page directory entries
    for (int i = 0; i < 1024; i++) {
        if (i == 0)
            page_directory[i] = ((unsigned long)page_table) | 3; // First 4MB mapped
        else
            page_directory[i] = 0; // Not present
    }

    Console::puts("Constructed Page Table object\n");
}
```

Figure 3: `PageTable()` Implementation

- **load():** Sets this PageTable instance as the current active page table by updating the `current_page_table` pointer and loading the page directory base into the `CR3` register. This ensures that the CPU uses this page table for all subsequent memory accesses.

```
/*------------------------------------------------------------*/
/* Load this page table (updates CR3) */
/*------------------------------------------------------------*/
void PageTable::load()
{
    current_page_table = this;
    write_cr3((unsigned long)page_directory);
    Console::puts("Loaded page table\n");
}
```

Figure 4: `load()` Implementation

- **enable_paging():** Activates the paging mechanism by setting the PG (paging) bit in the `CR0` control register. The `paging_enabled` variable is set to `1` to indicate that paging is turned on. The most significant bit in the `CR0` register corresponds to the paging enable flag; when set, it signals the processor to begin translating virtual addresses into physical addresses using the paging mechanism. After this, all memory accesses are interpreted as virtual addresses, and the CPU uses the loaded page tables for translation.

```
/*------------------------------------------------------------*/
/* Enable paging on the CPU */
/*------------------------------------------------------------*/
void PageTable::enable_paging()
{
    paging_enabled = 1;
    write_cr0(read_cr0() | 0x80000000); // Set PG bit in CR0
    Console::puts("Enabled paging\n");
}
```

Figure 5: `load()` Implementation

- **handle_fault():** Manages page faults dynamically. When a fault occurs, the function reads the faulting address from the `CR2` register and extracts the page directory and page table indices. If the corresponding page table does not exist, it allocates a new page table frame from the kernel memory pool and initializes all entries as not present. Then, it allocates a physical frame from the process memory pool for the missing page, updates the Page Table Entry (PTE) with present and writable flags, and refreshes the TLB by reloading `CR3`. This ensures that the faulting memory access can proceed correctly and that the system dynamically maintains valid page tables.

```
/*--------------------------------------------------------------*/
/* Handle a page fault exception */
/*--------------------------------------------------------------*/
void PageTable::handle_fault(REGS * _r)
{
    unsigned long faulting_address = read_cr2();  // Address that caused the fault
    unsigned long error_code = _r->err_code;

    Console::puts("Page fault occurred\n");

    // Extract directory and table indices
    unsigned long dir_index  = (faulting_address >> 22) & 0x3FF;
    unsigned long table_index = (faulting_address >> 12) & 0x3FF;

    // Get current page directory
    unsigned long * directory_base = (unsigned long *)(read_cr3());

    unsigned long dir_entry = directory_base[dir_index];
    unsigned long * page_table_ptr;

    // Check if directory entry is valid
    if (!(dir_entry & 1)) {
        // Allocate a new page table from kernel memory
        unsigned long new_table_frame = kernel_mem_pool->get_frames(1);
        page_table_ptr = (unsigned long *)(new_table_frame * PAGE_SIZE);

        // Initialize all entries as not present
        for (int i = 0; i < 1024; i++) {
            page_table_ptr[i] = 0;
        }

        // Update directory entry
        directory_base[dir_index] = ((unsigned long)page_table_ptr) | 3;
    } else {
        // Retrieve existing page table base
        page_table_ptr = (unsigned long *)(dir_entry & 0xFFFFF000);
    }

    // Allocate a new physical frame for the missing page
    unsigned long new_frame = process_mem_pool->get_frames(1);
    page_table_ptr[table_index] = (new_frame * PAGE_SIZE) | 3;  // Present + Writable

    // Refresh TLB
    write_cr3(read_cr3());

    Console::puts("Handled page fault successfully\n");
}
```

Figure 6: `handle_fault()` Implementation

Files Used/Modified

The following source files were involved in implementing the paging system:

- `page_table.C` – Implements the paging logic and related functions.
- `page_table.H` – Header file defining the `PageTable` class and related constants.
- `cont_frame_pool.H` and `cont_frame_pool.o` – Professor-provided files for implementing the `ContFramePool` class and frame management structures. The original `cont_frame_pool.C` has been removed; only the header file and precompiled object file (`.o`) are used to achieve successful implementation in MP3.

Makefile Changes:

- The following lines were deleted from the Makefile:

```
# ==== MEMORY =====
cont_frame_pool.o: cont_frame_pool.C cont_frame_pool.H
    $(GCC) $(GCC_OPTIONS) -c -o cont_frame_pool.o cont_frame_pool.C
```

- Note: Running `make clean` deleted the precompiled `cont_frame_pool.o` file. After this, the file was be copied back manually to allow successful compilation by running `make & make run` .

Overall, these changes ensure that the project uses the professor-provided `cont_frame_pool.H` and `.o` file while maintaining a working demand-paging system capable of translating virtual addresses, managing page tables, and handling page faults dynamically.

Testing

To verify the correct implementation of the demand-paging virtual memory system, we compiled and executed the kernel with the following commands:

```
make clean && make
make run
```

The implemented paging system was tested using the main kernel routine defined in `kernel.C`. The test sequence involves initializing the system, enabling paging, generating memory accesses that trigger page faults, and verifying that memory operations succeed. The following describes the steps and observations:

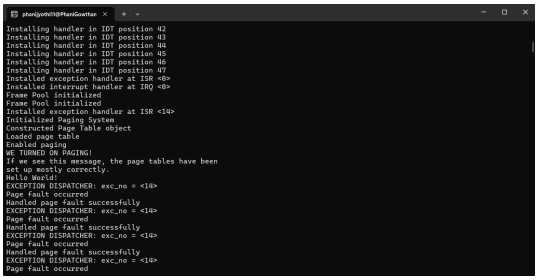
1. **System Initialization:**
   - Global Descriptor Table (GDT), Interrupt Descriptor Table (IDT), and exception/interrupt handlers are initialized.
   - Timer interrupts are set up using `SimpleTimer` and registered with the interrupt dispatcher.
   - The kernel memory pool and process memory pool are initialized using the `ContFramePool` class.
   - A memory hole is marked inaccessible to emulate real system constraints.
2. **Paging Setup:**
   - The page fault handler is registered at ISR 14 to capture page faults.
   - `PageTable::init_paging()` initializes the kernel and process memory pools and sets up shared memory size.
   - A `PageTable` object is constructed, which allocates memory for the page directory and the first page table.
   - The first 4 MB region is identity-mapped, and other page directory entries are marked as not present.
   - Paging is enabled via `PageTable::enable_paging()`, setting the PG bit in `CR0` and marking `paging_enabled` as true.
3. **Memory Access Test:**
   - Memory writes are performed at the address `FAULT_ADDR`, generating page faults for frames that are not yet allocated.
   - Each page fault triggers `PageTable::handle_fault()`, which:
     (a) Calculates the page directory and table indices from the faulting address.
     (b) Allocates a new page table frame from the kernel memory pool if needed.
     (c) Allocates a physical frame from the process memory pool for the missing page.
     (d) Updates the Page Table Entry (PTE) with the present and writable flags.
     (e) Refreshes the TLB by reloading `CR3`.
4. **Verification:**
   - After all memory writes, the kernel prints `DONE WRITING TO MEMORY. Now testing...` indicating the start of verification.

- If all values match the expected values, the test prints `TEST PASSED`.
- Otherwise, a failure message is printed indicating which memory access failed.

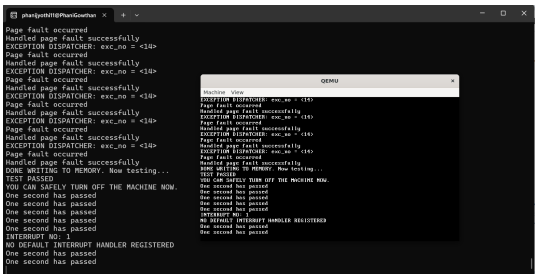
5. **Observations from Output:**

   - Messages such as `Initialized Paging System`, `Constructed Page Table object`, `Loaded page table`, and `Enabled paging` confirm proper setup.
   - Repeated messages `Page fault occurred` and `Handled page fault successfully` indicate correct dynamic handling of page faults.
   - The final message `TEST PASSED` confirms that all memory accesses were correctly handled, and the paging mechanism functions as expected.

This test ensures the `PageTable` class and the underlying memory pool management handle both initialization and dynamic page fault resolution correctly in a simulated kernel environment.



(a) Page table initialization output



(b) Page fault handling output

Figure 7: Test results showing page table setup and page fault handling in action.

Overall, the tests demonstrate that the paging subsystem correctly sets up virtual-to-physical memory mapping and dynamically handles page faults.